IMEN574 / IDSC574 · PROGRAMMING FOR AX

4

Coding Agents

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Coding Agents

01 **Use the Coding Agent**

Carry the report task through tools

02 **Connect Public Documentation**

Add and call one MCP server

03 **Finish the Workflow**

Save artifacts and bounded handoffs

01 · Use the Coding Agent

01 **Use the Coding Agent**

Carry the report task through tools

02 **Connect Public Documentation**

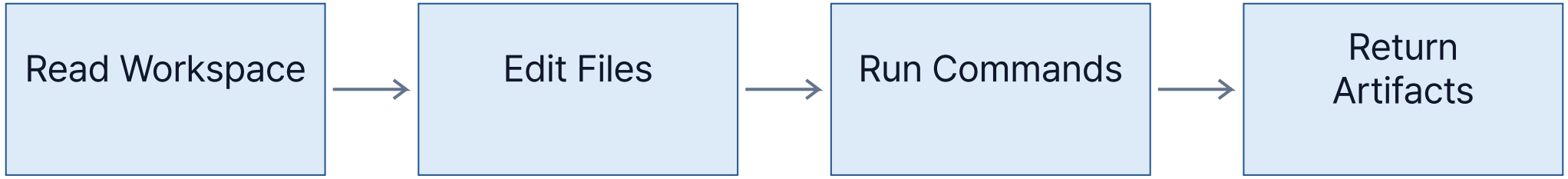
Add and call one MCP server

03 **Finish the Workflow**

Save artifacts and bounded handoffs

Why a Coding Agent

Let the assistant carry a bounded task across read, edit and execution



The agent can act across the whole small task

The Same Report Task

A familiar artifact keeps attention on the new workflow

The starting workspace

README.md and inspection.csv

Unchanged checks.py

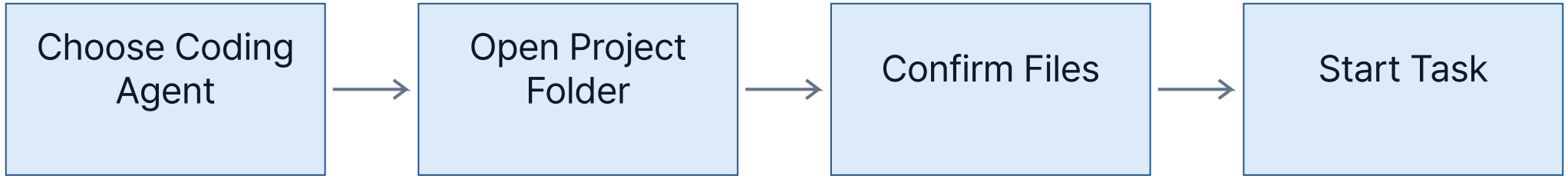
Working app.py from the editor session

A report ready for one small improvement

Reuse the report instead of introducing a new programming problem

Open an Agent Session

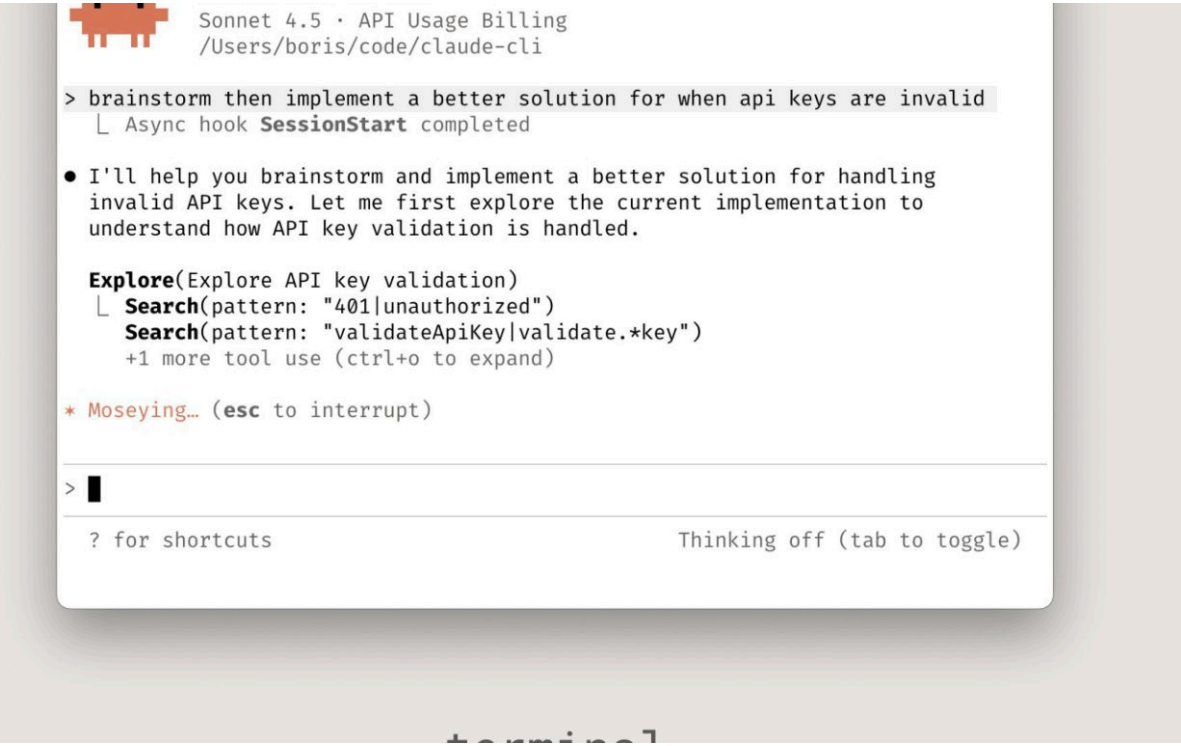
Select the task folder before requesting action



The selected folder defines where the work happens

One Agent, Several Interfaces

A terminal or editor can expose the same task workspace



The interface should reveal the current task and actions

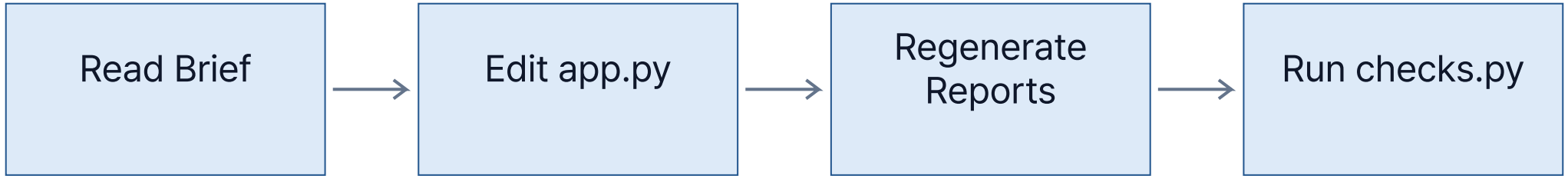
A Read-First Request

Ask the agent to inspect before changing the project

```
Read README.md, inspection.csv, checks.py and app.py
Explain the two output files and the run command
List the files you would change to improve HTML readability
Do not edit or run commands yet
```

Inspect the Proposed Plan

A short plan should name concrete files and outputs



The plan stays inside the existing report workspace

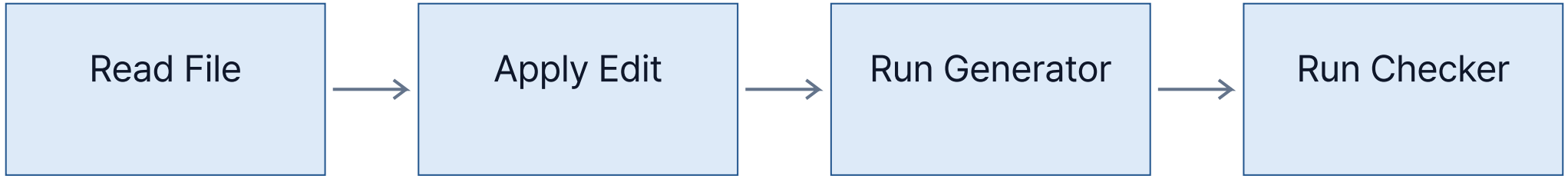
Authorize a Small Build

Give the agent a complete but narrow task

```
Improve the HTML report with a quality-status column
Show valid, missing and invalid rows distinctly
Keep JSON values, raw-row order and both A06 rows
Edit app.py only; do not change input or checks.py
Run the generator and checks.py; return the artifact paths
```

Read, Edit, Run, Test

Inspect the actual tool calls as the task proceeds



Actions leave observable files and process results

Demo: Execute a Tool Sequence

Use the local workspace to read, analyze, save and inspect a report

Discover the tools; produce report.json with threshold 0.5, then read it back

Tool call JSON

```
{
  "tool": "read_file",
  "arguments": {
    "path": "inspection.csv"
  }
```

Execute tool | Reset workspace

```
Executed: read_file
id,score
A01,0.20
A02.0.50
```

Local functions execute each call; the AI client connects to MCP separately

Answer: A Traceable Tool Result

The final read confirms the artifact, not just an intention

```
{"tool":"analyze_scores","arguments":{"threshold":0.5}}
{"tool":"write_report"}
{"tool":"read_report"}

# raw: 8, valid: 6, flagged: 4, threshold: 0.5
```

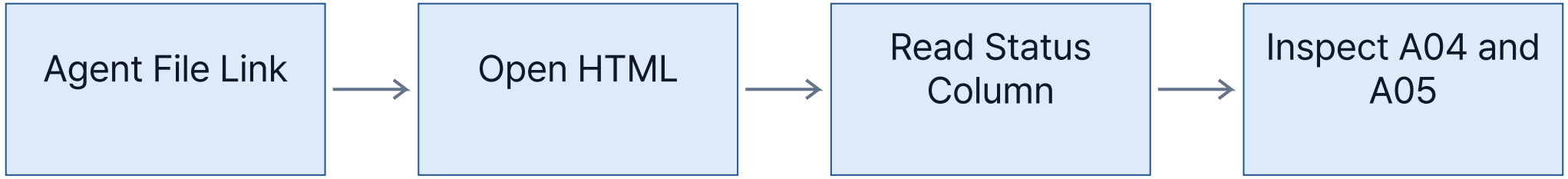
Command Approval

Review a proposed command before allowing its effect

Proposed action	This task
Read inspection.csv	Needed
Run local generator	Needed
Delete source CSV	Outside scope
Upload private data	Outside scope

Open the Returned Artifact

A path in the response should lead to the actual report



The returned file must contain the requested improvement

An Execution Record

Record what ran rather than what the agent intended

Actual work record

The generator command

The checker output

Changed file names

The opened report path

The record connects actions to the saved artifact

02 · Connect Public Documentation

01 Use the Coding Agent

Carry the report task through tools

02 **Connect Public Documentation**

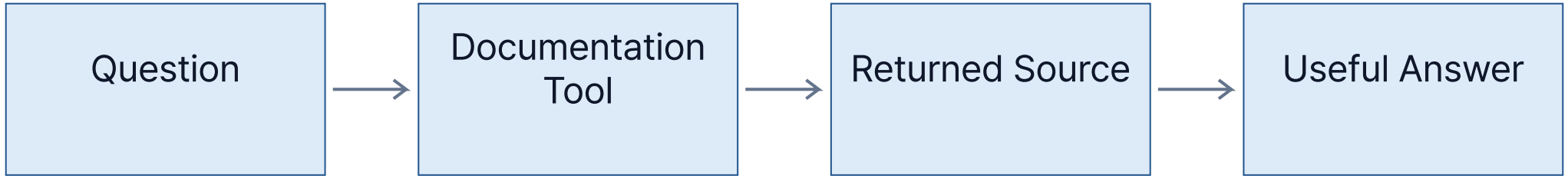
Add and call one MCP server

03 Finish the Workflow

Save artifacts and bounded handoffs

Why Connect a Documentation Tool

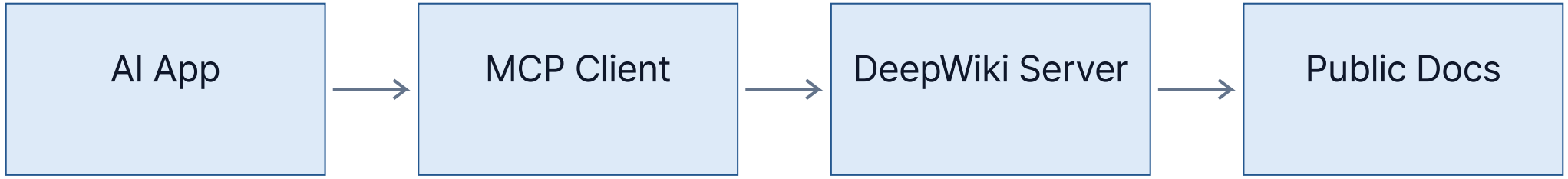
A tool can supply context beyond the local project files



A connected source extends the available task context

MCP in This Exercise

Model Context Protocol (MCP) connects the client to server capabilities

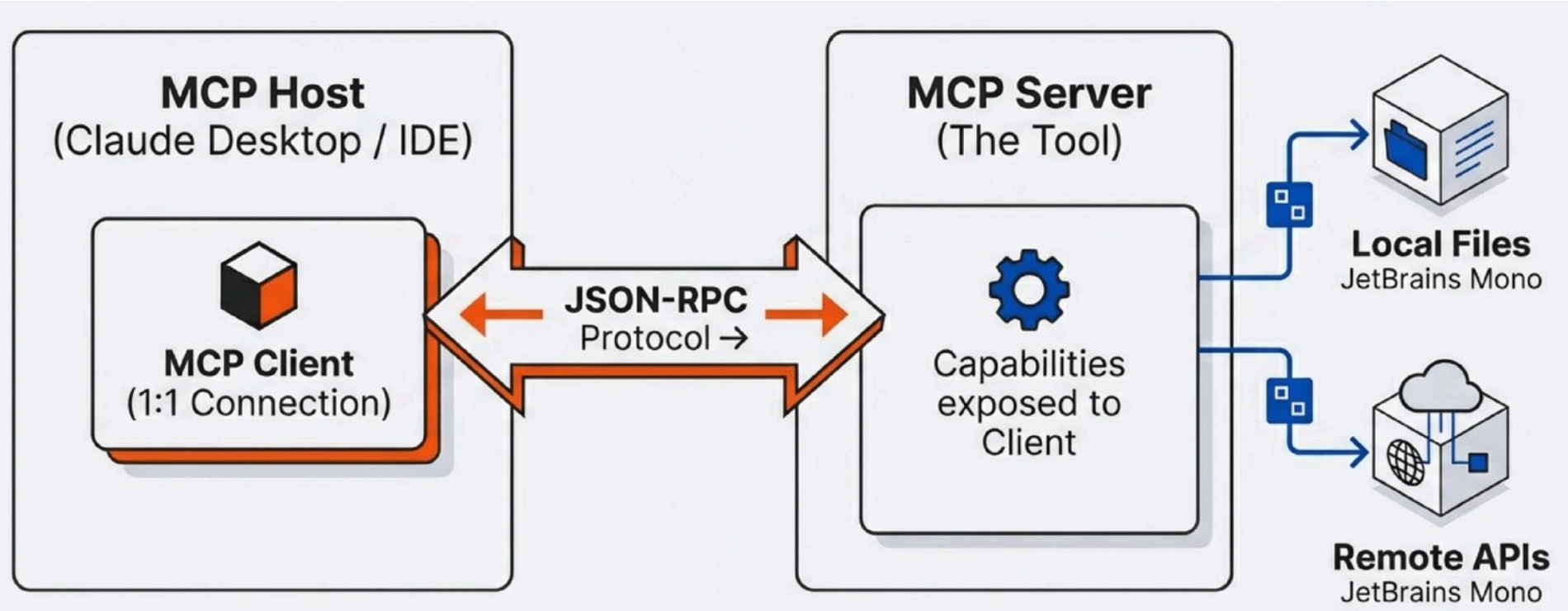


The exercise connects one app to one public documentation server

https://docs.devin.ai/work-with-devin/deepwiki-mcp

The Connection Boundary

The server exposes capabilities through a defined interface



The client-server boundary makes capabilities discoverable

A Public Read-Only Source

DeepWiki supplies documentation about public repositories

DeepWiki MCP

Publisher: Cognition / Devin documentation

Service: public repository documentation

Endpoint: https://mcp.deepwiki.com/mcp

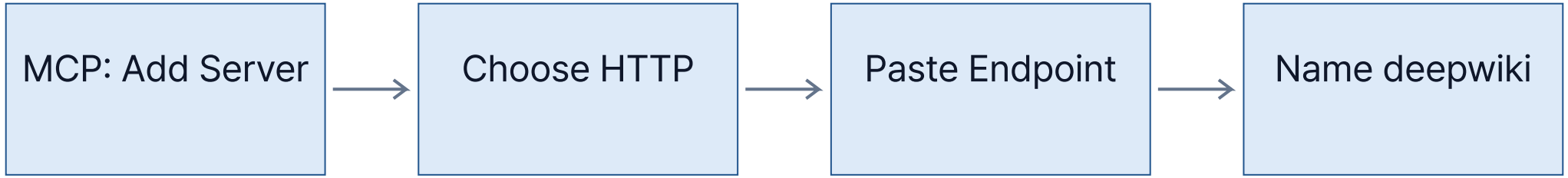
No private project files are needed for the exercise

The tutorial needs only a public repository name

https://docs.devin.ai/work-with-devin/deepwiki-mcp

Add the Server

Use the client’s guided HTTP server setup



https://code.visualstudio.com/docs/agent-customization/mcp-servers

Connection and Trust

Check the publisher, endpoint and offered capabilities

Check	Expected target
Publisher docs	docs.devin.ai
Endpoint host	mcp.deepwiki.com
Transport	HTTP /mcp
Exercise payload	Public repository name

Discover the Tools

Inspect the tool list rather than guessing a tool name

Tool	Role
read_wiki_structure	List documentation topics
read_wiki_contents	Read repository documentation
ask_question	Answer a repository question

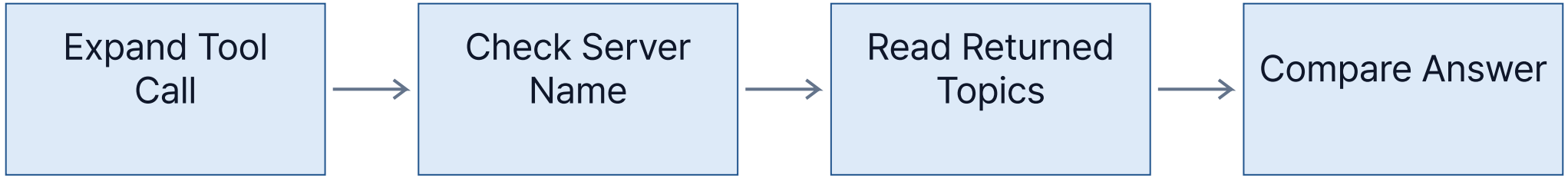
Make One Read Call

Request documentation structure for a public repository

```
Use the deepwiki MCP server
Call read_wiki_structure for the public repo python/cpython
Use the actual tool schema shown by the client
Do not change files
Report the tool name and returned documentation topics
```

Inspect the Returned Content

Open the tool-call detail before accepting the answer



https://code.visualstudio.com/docs/agent-customization/mcp-servers

Tool Evidence and Model Text

A fluent answer does not demonstrate that a tool was called

Evidence	Inspect
Model says it read a file	A claim to verify
Tool trace contains read_file	Arguments and returned content
Saved report is read back	Actual artifact and source-derived counts

A Follow-Up Documentation Question

Use the returned source to narrow the next request

```
Using the returned documentation, locate CSV parsing guidance
Explain how quoted commas are handled
Cite the returned source and identify any unanswered detail
Do not edit the inspection project
```

A Failed Connection

Fix the connection layer before rewriting the task prompt

Observation	Next check
Server absent	URL and scope
Startup failure	Client log and HTTP support
No tool call	Enabled tools and explicit server name
Repository unavailable	Public indexing and repo spelling

https://code.visualstudio.com/docs/agent-customization/mcp-servers

03 · Finish the Workflow

01 Use the Coding Agent

Carry the report task through tools

02 Connect Public Documentation

Add and call one MCP server

03 **Finish the Workflow**

Save artifacts and bounded handoffs

The Finished MCP Exercise

Keep the real call evidence with the source URL

Connection work record

Client and server name

Actual discovered tool names

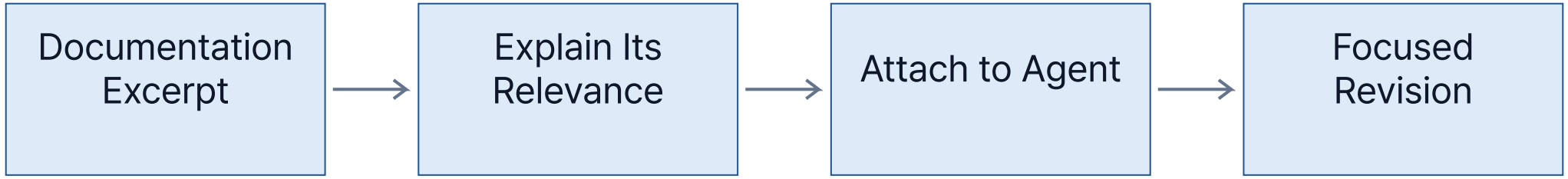
One public-repo tool call

Returned excerpt or the observed connection error

The record identifies what the client actually called

Reuse Documentation in the Project

Bring a relevant source back to the report task



A connected source should help the current task

Project Instructions

Keep the acceptance rule close to the files

A project instruction

Read README.md before editing

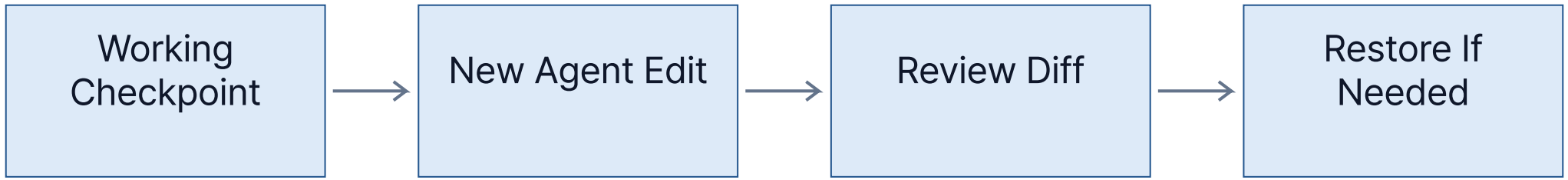
Keep inspection.csv and checks.py unchanged

Use the standard-library generator command

Return artifact paths and actual check output

Checkpoint Recovery

Recover a local edit before broadening the task



https://code.visualstudio.com/docs/agents/run/review-code-edits

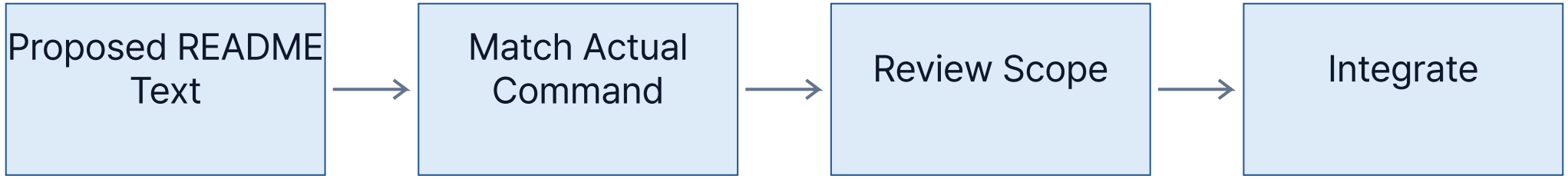
A Bounded Delegation

Delegate one artifact while retaining the project contract

```
In a separate task, draft README usage instructions
Use the existing run command and output filenames
Return a proposed text; do not edit project files
Do not invent a successful execution result
```

Review the Handoff

Check one delegated result before integrating it



The handoff is accepted against the same project files

What the User Controls

Task, context, tool scope and completion remain explicit choices

Control	In this exercise
Task	Improve one report
Context	Project files and public docs
Scope	Local edits; documentation reads
Completion	Opened artifact and actual checks

Takeaway: Coding Agents

01 Give the coding agent a complete bounded file task

02 Connect one documentation server and inspect a real tool call

03 Keep the artifacts and the actual execution record